

HEALTHY BITE

Digital Restaurant Menu & Food Ordering System

Complete Project Understanding, Architecture, File Guide, Local Execution & Viva Preparation Guide

Executive Summary for Beginners: This comprehensive technical manual is generated directly from the Healthy Bite production codebase. It details the custom Model-View-Controller (MVC) architecture, the zero-dependency PSR-4 autoloader, the 17-table relational MySQL schema, the real-time nutrition calculation engine, QR token security validation, local XAMPP execution, and step-by-step viva voce defense.

1. Technology Stack & Core Identity

Component	Technology / Implementation	Architectural Role in Healthy Bite
Backend Core	PHP >= 8.0 (Strict Typing)	Server-side request processing, security guards, session management.
Database	MySQL / MariaDB (InnoDB, utf8mb4)	Stores 17 relational tables with foreign keys and cascade integrity.
Data Layer	PDO (PHP Data Objects)	Singleton connection, prepared statements, SQL injection immunity.
Architecture	Custom Lightweight MVC	Front Controller, dynamic regex router, repository pattern.
Autoloading	Custom PSR-4 Fallback Autoloader	Zero external dependencies. Runs standalone without Composer vendor.
Frontend UI	HTML5 + Modular CSS3 + ES6 JS	Fast, responsive customer menu, live modals, and floating cart.
Auth Engine	Bcrypt (Cost 12) + PHP Sessions	Session fixation regeneration, role-based isolation (Owner/Staff/Admin).

2. System Architecture & Request Lifecycle

Every HTTP request entering Healthy Bite travels through a clean, deterministic pipeline:

- Web Browser (Diner scans QR or Owner logs in)
- Apache / Web Server receives request -> .htaccess routes silently to public/index.php
- public/index.php initializes App\Core\App -> Session::start() -> PSR-4 Autoloader

4. App\Core\Router matches URL regex against routes/web.php or routes/api.php
5. Controller executes -> delegates business rules to Domain Services (Pricing, Nutrition, QR)
6. Repositories execute safe parameterized PDO queries against MySQL
7. Controller renders HTML view template (wrapped in Layout) or returns JSON API response
8. Browser receives response -> JavaScript handles live updates without full reloads

3. Directory Structure & Key Responsibilities

Directory	Key Responsibilities & Contents	Safety Level
app/Core/	The engine: App.php, Router.php, Database.php, Request.php, Response.php, Session.php.	DO NOT TOUCH
app/Controllers/	Request handlers: MenuController, OrderController, Owner/*, Admin/*.	EDIT CAREFULLY
app/Repositories/	Database query layer: FoodRepository, OrderRepository, RestaurantRepository.	EDIT CAREFULLY
app/Services/	Pure business calculations: NutritionService, PricingService, QrService, CartService.	EDIT CAREFULLY
config/	Global application constants, tax rates, role IDs, and PDO database configs.	DO NOT TOUCH
public/	Web root: index.php (entry point) + assets/ (CSS stylesheets, JS logic, images).	SAFE (Assets)
resources/views/	HTML presentation: customer/menu.php, owner/kitchen_orders.php, layouts/.	SAFE TO EDIT
routes/	URL route registries: web.php (browser endpoints) and api.php (JSON endpoints).	EDIT CAREFULLY

4. Relational Database Schema (All 17 Tables)

Healthy Bite utilizes a strictly normalized MariaDB/MySQL database schema containing 17 relational tables:

Table Name	Primary Key	Key Foreign Keys & Relational Purpose
restaurants	id	Master entity for dining establishments; status (pending, approved, rejected).
branches	id	FK: restaurant_id. Multi-branch physical locations.
restaurant_tables	id	FK: branch_id. Physical dining tables (T-01, T-02) with capacity and status.
qr_tokens	id	FK: table_id. Cryptographic tokens (e.g., hb_greenhouse_downtown_t1_sec).
categories	id	FK: restaurant_id. Food groupings (Bowls, Smoothies, Warm Plates).
food_items	id	FK: restaurant_id, category_id. Base price, calories, protein, carbs, fat, fiber.
food_variants	id	FK: food_item_id. Portion adjustments (Regular vs Large Athlete Portion).
food_customizations	id	FK: food_item_id. Add-ons (Extra Grilled Tofu, Olive Oil Dressing).
customers	id	Customer contact ledger (Name, Mobile, Email).
orders	id	FK: restaurant_id, branch_id, table_id, customer_id. Order number, totals, status.
order_items	id	FK: order_id, food_item_id. Frozen historical snapshots of price & macros.
order_item_customizations	id	FK: order_item_id, customization_id. Snapshot of selected add-ons.
payments	id	FK: order_id. Payment records (card, upi, cash, net_banking) & status.
reviews	id	FK: restaurant_id, order_id. Ratings (1-5) and feedback.
roles	id	Role definitions (1: Super Admin, 2: Owner, 3: Manager, 4: Staff).
users	id	FK: role_id, restaurant_id. Staff and Owner login accounts with Bcrypt.
admin	id	System administration and platform governance entity.

5. Domain Calculation Engines

The Nutrition Calculation Engine (NutritionService.php):

Nutritional metrics are computed using a strict mathematical formula across 8 nutrient fields: *calories*, *protein*, *carbs*, *fat*, *fiber*, *sugar*, *sodium*, and *caffeine*.

$$\text{Final Nutrient} = \text{Base Nutrient} + \text{Variant Adjustment} + \sum (\text{Customization Adjustment} \times \text{Quantity})$$

NULL Data Integrity Rule: If a nutrient value was never provided by the restaurant, the system strictly preserves NULL rather than converting it to a deceptive '0'. Calories are rounded to integer units, while macronutrients are rounded to 2 decimal places.

The Dynamic Pricing Engine (PricingService.php):

Financial billing calculations follow a deterministic 3-tier calculation pipeline:

1. **Single Configured Item Price:** $\text{\$}\{\text{Base Price}\} + \{\text{Variant Price Adjustment}\} + \sum (\{\text{Customization Price}\} \times \{\text{Qty}\})\text{\$}$
2. **Line Total:** $\text{\$}\{\text{Single Item Price}\} \times \{\text{Food Quantity}\}\text{\$}$
3. **Order Bill Totals:** $\text{\$}\{\text{Subtotal}\} + \{\text{Tax (5\%)}\} + \{\text{Service Charge (0\%)}\} = \{\text{Total Amount Payable}\}\text{\$}$

6. Operational Workflows: Kitchen Live Orders (KDS Kanban)

Healthy Bite includes a real-time Kitchen Display System (KDS) located at /owner/kitchen-orders. Orders advance across a synchronized Kanban board:

Status Stage	Kitchen Meaning	Trigger / Transition
placed	New order received from customer table QR.	Chef clicks 'Accept Order' button.
preparing	Kitchen staff actively cooking the meal.	Chef clicks 'Mark as Ready' button.
ready	Meal plated; waiting for service to table.	Service staff serves meal, clicks 'Complete'.
completed	Diner served; order archived into daily sales.	Settled and finalized.

7. Localhost Execution & Testing Guide

Prerequisites: XAMPP (Apache + MariaDB/MySQL) + PHP >= 8.0.

Step 1: Start Apache and MySQL in XAMPP Control Panel (verify green status).

Step 2: Open <http://localhost/phpmyadmin/>, create database healthy_bite, and import database/healthy_bite_infinityfree_dump.sql.

Step 3: Configure .env with DB_HOST=127.0.0.1, DB_DATABASE=healthy_bite, and DB_USERNAME=root.

Step 4: Start the built-in server in project root: `php -S localhost:8000 -t public`.

Step 5: Open http://localhost:8000/qr/hb_greenhouse_downtown_t1_sec to test customer QR ordering.

Demo Credentials (All Passwords: Password@123):

Portal Role	Login URL	Demo Email	Password
Restaurant Owner	/owner/login	owner@greenhousekitchen.com	Password@123
Kitchen Staff	/owner/login	aarav@greenhouse.in	Password@123
Platform Super Admin	/admin/login	admin@healthybite.com	Password@123

8. Viva Voce Examination Defense

Q: Why custom MVC instead of Laravel?

A: Demonstrates full mastery of foundational computer science principles: the Front Controller pattern, PSR-4 autoloading, regular expression routing, and PDO abstraction without framework bloat.

Q: How is SQL Injection prevented?

A: By using PDO prepared statements with strict parameter binding (:id, :token) across all Repository classes. User inputs are never concatenated directly into SQL queries.

Q: How does the QR token prevent tampering?

A: The QR code embeds a random, high-entropy token (`/qr/{token}`) resolved via multi-table inner joins on the server. The server overrides client parameters with verified database context.

Q: Why store snapshots in order items?

A: To preserve financial and nutritional history. If an owner later updates dish prices or ingredients, past receipts and tax audits remain 100% accurate.